Chonnx

Motor de inferencia ONNX genérico para ChucK

Versión 1.0 · 2026

Chonnx es un chugin para ChucK que expone un motor de inferencia ONNX Runtime completamente genérico. Permite cargar cualquier modelo **.onnx** de entrada/salida float32 y ejecutarlo en tiempo real dentro de un patch de ChucK, sin modificar el chugin para cada arquitectura. El usuario define la lógica musical (muestreo, secuenciación, síntesis) en ChucK; Chonnx se ocupa del tensor I/O.

1. Arquitectura general

Chonnx actúa como una capa de transporte entre ChucK y ONNX Runtime. Su diseño es deliberadamente agnóstico: no asume el número de inputs ni outputs, ni el significado semántico de cada tensor. Todo eso lo decide el modelo y el código ChucK que lo orquesta.

Capa	Responsabilidad
Modelo .onnx	Arquitectura de red neuronal entrenada en Python (PyTorch, Keras, etc.). Cualquier tipo: GRU, LSTM, Transformer, MLP, CNN.
Chonnx (C++)	Carga el modelo con ONNX Runtime, gestiona los buffers de entrada y salida, ejecuta la inferencia en respuesta a run().
Código ChucK	Define la lógica musical: prepara los inputs, llama a run(), lee los outputs, muestrea probabilidades, agenda notas.

Flujo de datos en tiempo de ejecución:

```
[ChucK] set_input(idx, val) → buffer de entrada
[ChucK] run() → ONNX Runtime ejecuta la red
[ChucK] get_output(idx) ← buffer de salida (probabilidades / valores)
[ChucK] argmax / muestreo → decisión musical (pitch, duración, etc.)
```

2. API del chugin

Chonnx expone seis métodos públicos. Los métodos **set_input** y **get_output** están sobrecargados: la versión de dos argumentos opera sobre el input/output 0 (compatibilidad con modelos de entrada única), y la de tres argumentos soporta arquitecturas multi-tensor.

Método	Firma	Descripción
load_model	load_model(string path)	Carga el archivo .onnx indicado. Auto-detecta número de inputs/outputs, nombres de nodos y shapes. Inicializa los buffers internos.
set_input (single)	set_input(int idx, float val)	Escribe val en la posición idx del input 0. Para modelos de un solo tensor de entrada.
set_input (multi)	set_input(int in, int idx, float val)	Escribe val en la posición idx del tensor de entrada número in. Necesario cuando el modelo tiene varios inputs (p.ej. DDSP: audio + pitch).
run	int run()	Ejecuta la inferencia. Devuelve 1 si tiene éxito, 0 si ocurre un error. Tras esta llamada, get_output devuelve los resultados actualizados.
get_output (single)	float get_output(int idx)	Lee el valor en la posición idx del output 0. Para modelos de un solo tensor de salida.
get_output (multi)	float get_output(int out, int idx)	Lee el valor en la posición idx del tensor de salida número out.

> **Nota sobre dimensiones dinámicas:** si el modelo declara una dimensión como -1 (batch size dinámico), Chonnx la trata como 1 en tiempo de ejecución. El wrapper de exportación debe proyectar la entrada al shape esperado antes de exportar.

3. Requisitos del modelo .onnx

Chonnx impone los siguientes requisitos sobre el archivo .onnx:

Requisito	Detalle
Tipo de dato	float32 en todos los tensores de entrada y salida.
Formato de exportación	ONNX estándar (opset 11 o superior). No TorchScript (.jit), no TensorFlow SavedModel.
Softmax en output	Si el modelo produce probabilidades, aplicar softmax en el wrapper de exportación, no dentro de la red de entrenamiento. Esto evita pasar probabilities a CrossEntropyLoss durante training.
Shapes fijos o dinámicos	Ambos son compatibles. Los dims dinámicos (-1) se resuelven a 1.
Batch dimension	Siempre batch=1. Chonnx no soporta inferencia por lotes.
Pesos embebidos	Usar external_data=False al exportar desde PyTorch para que todos los pesos queden en el .onnx. Evita el archivo sidecar .onnx.data que falla en rutas de Windows.

4. Flujo de trabajo: Python → ONNX → ChucK

4.1 Entrenamiento en Python

Define la arquitectura de red en PyTorch (u otro framework), entrena con los datos de tu dominio, y guarda solo los pesos. **No apliques softmax en el forward() de entrenamiento** — CrossEntropyLoss espera logits crudos.

```
class MiModelo(nn.Module):
def __init__(self):
super().__init__()
self.gru = nn.GRU(INPUT_SIZE, HIDDEN_SIZE, num_layers=2,
batch_first=True, dropout=0.3)
self.fc = nn.Linear(HIDDEN_SIZE, OUTPUT_SIZE)
def forward(self, x): # x: [batch, seq, features]
out, _ = self.gru(x)
return self.fc(out[:, -1, :]) # logits crudos

# Entrenamiento con logits crudos
loss = nn.CrossEntropyLoss()(outputs, targets)

# Guardar pesos
torch.save(model.state_dict(), 'mi_modelo_weights.pth')
```

4.2 Exportación a ONNX (wrapper de exportación)

Crea un wrapper separado que: (a) recibe la entrada en formato plano [1, SEQ_LEN × FEATURES] para simplificar el lado ChucK, (b) reshape internamente, y (c) aplica softmax sobre los logits antes de exportar.

```
class Wrapper(nn.Module):
def __init__(self, base):
super().__init__()
self.model = base
def forward(self, x_flat): # x_flat: [1, SEQ_LEN * INPUT_SIZE]
x = x_flat.view(1, SEQ_LEN, INPUT_SIZE)
logits = self.model(x)
return torch.softmax(logits, dim=-1) # probabilidades para ChucK

wrapped = Wrapper(base)
dummy = torch.zeros(1, SEQ_LEN * INPUT_SIZE)
torch.onnx.export(
wrapped, dummy, 'mi_modelo.onnx',
input_names=['input'],
output_names=['output'],
opset_version=17,
external_data=False # ← importante en Windows
)
```

4.3 Uso en ChucK

Carga el modelo, rellena el buffer de entrada con el contexto de las últimas notas, llama a run() y lee las probabilidades del output. El muestreo (argmax o temperatura) se implementa directamente en ChucK.

```
Chonnx brain;
brain.load_model(me.dir() + "mi_modelo.onnx");

// Llenar contexto: últimas SEQ_LEN notas [pitch_norm, delta_norm]
for (0 => int i; i < SEQ_LEN * 2; i++) {
brain.set_input(i, context[i]);
}

brain.run();

// Leer probabilidades de pitch (primeros 128 valores)
0 => int best; 0.0 => float best_p;
for (0 => int i; i < 128; i++) {
brain.get_output(i) => float p;
if (p > best_p) { p => best_p; i => best; }
}
// best = nota MIDI predicha
```

5. Ejemplo: JazzSoloGRU

El modelo **jazz_solo_brain.onnx** es el primer modelo entrenado sobre Chonnx. Se trata de una GRU de 2 capas entrenada sobre el dataset WJazzD (transcripciones de solos de jazz de músicos reales). Predice la siguiente nota y su duración dado el contexto de las 8 notas anteriores.

Hiperparámetros de entrenamiento

Parámetro	Valor	Descripción
SEQ_LEN	8	Ventana de contexto: las últimas 8 notas alimentan la GRU.
INPUT_SIZE	2	Características por timestep: pitch normalizado [0–1] y delta-time normalizado.
HIDDEN_SIZE	128	Dimensión del estado oculto de la GRU. Define la capacidad del modelo.
num_layers	2	GRU apilada de 2 capas. Permite jerarquías temporales.
dropout	0.3	Regularización entre capas GRU para evitar sobreajuste.
OUTPUT_PITCH	128	Logits para las 128 notas MIDI. Se convierten en probabilidades al exportar.
OUTPUT_DUR	8	Logits para 8 categorías de duración: desde fusa hasta redonda y más.
FLAT_SIZE	16	Tamaño del tensor de entrada plano (SEQ_LEN × INPUT_SIZE = 8 × 2).
Batch	512	Muestras por paso de gradiente durante entrenamiento.

Parámetro	Valor	Descripción
Épocas	50	Ciclos completos sobre el dataset. LR se reduce a la mitad cada 10 épocas.
Optimizador	Adam lr=0.001	Adam con gradient clipping (max norm 1.0) para estabilizar el entrenamiento.

Formato de los tensores

```
Input → 'input_note_state' shape [1, 16] float32
Índices 0..15 = 8 pasos × [pitch/127, min(delta/2, 1)]

Output → 'next_note_probabilities' shape [1, 136] float32
Índices 0..127 = probabilidades de pitch (suma = 1.0)
Índices 128..135 = probabilidades de duración (suma = 1.0)

Categorías de duración (índices 128..135):
0=fusa(<0.15s) 1=semicorchea(<0.30s) 2=corchea(<0.55s) 3=negra(<1.05s)
4=blanca(<2.05s) 5=redonda(<4.05s) 6=larga(>=4.05s) 7=muy larga
```

Datos de entrenamiento: WJazzD

El Weimar Jazz Database (WJazzD) contiene transcripciones MIDI de solos monofónicos de músicos de jazz grabados en condiciones reales. Las notas se extraen como secuencias de (pitch, inter-onset interval), sin información de compás ni acorde, lo que hace al modelo independiente del contexto armónico. La ruta de los archivos MIDI se configura en entrenar.py.

6. Estrategias de muestreo en ChucK

Una vez que run() devuelve las probabilidades, ChucK debe convertirlas en una decisión discreta (qué nota tocar). Hay tres estrategias principales:

Estrategia	Implementación en ChucK	Efecto musical
Argmax (greedy)	Iterar get_output(0..127) y quedarse con el índice de mayor valor.	Siempre elige la nota más probable. Frases predecibles, conservadoras.
Temperatura	Dividir los logits (antes de softmax) por T. T < 1 = más conservador, T > 1 = más exploratorio. Requiere exponer logits en el ONNX.	Controla el nivel de riesgo armónico en tiempo real.
Top-K	Poner a cero todas las probabilidades excepto las K mayores y renormalizar. Luego muestrear con Math.random2f().	Elimina opciones improbables pero mantiene diversidad. K=10 es un buen punto de partida.

7. Chonnx vs ChAR: cuándo usar cada uno

Chonnx y ChAR son complementarios. ChAR es un motor autoregresivo especializado para generación musical con contexto armónico; Chonnx es un motor de inferencia genérico. La elección depende del problema:

Característica	Chonnx	ChAR
Tipo de modelo	Cualquier ONNX float32	Dos modelos ONNX separados (pitch + duración)
Lógica de generación	En ChucK (el usuario la escribe)	Dentro del chugin (generate())
Contexto armónico	Debe implementarse en el modelo	Integrado: set_progression(), F_CHORD
Autoregresión	Manual en ChucK	Automática con generate()
Arquitecturas soportadas	GRU, LSTM, MLP, CNN, Transformer, DDSP...	GRU dual (fijo)
Curva de aprendizaje	Mayor (lógica en ChucK)	Menor (API de alto nivel)
Flexibilidad	Máxima	Optimizada para improvisación jazz
Caso de uso ideal	Modelos experimentales, ritmo, síntesis, control de efectos	Improvisación melódica sobre progresiones de acordes

8. Guía: construyendo tu propio modelo

Cualquier modelo cuyo dominio se pueda representar como una secuencia de vectores float32 es compatible con Chonnx. El siguiente checklist cubre los pasos necesarios:

Paso	Descripción
1. Definir inputs	¿Qué información necesita el modelo en cada paso de tiempo? Normaliza todos los valores a [0, 1] o [-1, 1]. Ejemplo: [pitch/127, delta/max_delta, beat_pos/beats_per_bar].
2. Definir outputs	¿Qué predice el modelo? Si son categorías, usa logits + CrossEntropyLoss en entrenamiento, softmax en exportación. Si es un valor continuo, usa MSELoss y no apliques softmax.
3. Elegir arquitectura	Para secuencias temporales: GRU o LSTM. Para relaciones no-temporales: MLP. Para patrones rítmicos locales: CNN 1D.
4. Preparar datos	Extrae secuencias de MIDI con mido o pretty_midi. Construye ventanas deslizantes de tamaño SEQ_LEN.
5. Entrenar	Usa PyTorch con logits crudos en el forward(). Guarda state_dict() al terminar.
6. Exportar	Crea un Wrapper que reciba input plano y aplique softmax. Exporta con external_data=False.
7. Validar	Verifica con onnxruntime en Python que output.shape y las sumas de probabilidades son correctas antes de pasar a ChucK.
8. Integrar en ChucK	brain.load_model(...), bucle set_input/run/get_output, lógica de muestreo y síntesis.

9. Decisiones de diseño

Decisión	Razón
API de un solo tensor (backward compat)	set_input(idx, val) opera sobre el tensor 0 para no romper código existente que usaba versiones anteriores del chugin con un solo input.
Soporte multi-tensor	Arquitecturas como DDSP requieren múltiples inputs independientes (p.ej. pitch + loudness + z). La sobrecarga de 3 argumentos los soporta sin cambiar la API de un solo tensor.
ONNX sobre TorchScript	ONNX es el formato de intercambio más portable entre frameworks. TorchScript (.jit) requiere libtorch, que pesa ~1 GB. ONNX Runtime pesa ~15 MB y funciona desde cualquier framework.
external_data=False	En Windows, la exportación con datos externos crea un sidecar .onnx.data cuya escritura falla con OSError (errno 22). Al embeber los pesos en el .onnx el archivo es autónomo y portable.

Decisión	Razón
Softmax en el wrapper, no en training	CrossEntropyLoss aplica su propio log_softmax internamente. Pasar probabilidades ya normalizadas (doble softmax) aplana los gradientes y hace que la pérdida no baje del baseline aleatorio (≈6.93 para 128+8 clases).
Dimensiones dinámicas resueltas a 1	Algunos exportadores declaran la batch dimension como -1. Chonnx la colapsa a 1 para garantizar inferencia muestra a muestra, que es el modo natural de uso en tiempo real.